

IDENTIFICAÇÃO DO GRUPO (4 a 5 integrantes)

Integrante 1: RA:
Integrante 2: RA:
Integrante 3: RA:
Integrante 4: RA:
Integrante 5: RA:
Turma: Data de entrega: ____/____/____

1 Objetivo

Desenvolver, em grupo de 4 a 5 integrantes, um **Analisador de Sequências de DNA** em Linguagem C que permita cadastrar, listar, analisar e comparar sequências genéticas. O sistema é uma versão simplificada de ferramentas reais de bioinformática e aplica todo o conteúdo da disciplina.

CONTEXTO BIOLÓGICO

O **DNA** é a molécula que armazena a informação genética dos seres vivos. É formado por uma sequência de quatro bases: **Adenina (A)**, **Timina (T)**, **Citosina (C)** e **Guanina (G)**. Em programação, trabalhamos com essas sequências como **strings de caracteres**, por exemplo: ATGCCGTAACGGATTAG.

2 Fundamentação Teórica

Esta seção apresenta uma revisão dos principais conteúdos da disciplina e dos conceitos biológicos necessários para a realização do trabalho. **Leia com atenção antes de começar** — todos os tópicos aqui descritos serão cobrados.

2.1 Parte A — Conceitos Biológicos

O que é DNA

O DNA (ácido desoxirribonucleico) é uma molécula presente em todos os seres vivos. Imagine-o como uma “fita” formada por uma sequência de quatro “letras” (bases), em qualquer ordem:

- **A** — Adenina
- **T** — Timina
- **C** — Citosina
- **G** — Guanina

Uma sequência de DNA é apenas uma string formada por essas letras. Exemplo:

```
1 ATGCCGTAACGGATTAGCC
```

No nosso sistema, vamos tratar cada sequência como um vetor de caracteres (`char[]` em C). **Não há nada de mágico:** é só uma string que aceita apenas 4 caracteres possíveis.

Conteúdo GC

O **conteúdo GC** é o percentual de bases G e C em uma sequência. Por que isso importa? Porque diferentes organismos têm conteúdos GC característicos — humanos têm cerca de 41%, a bactéria *E. coli* tem 51%, e algumas bactérias termofílicas chegam a 70%.

Fórmula:

$$\text{GC\%} = \frac{(\text{quantidade de G} + \text{quantidade de C})}{\text{tamanho total}} \times 100$$

Exemplo passo a passo com a sequência ATGCCG (6 bases):

- Bases G: 2 (posições 3 e 6)
- Bases C: 2 (posições 4 e 5)
- Total GC: $2 + 2 = 4$
- $\text{GC\%} = 4/6 \times 100 = 66,67\%$

Motif

Um **motif** é um padrão curto de bases que tem significado biológico. Por exemplo:

- **TATA** — aparece próximo ao início de muitos genes (caixa TATA);
- **GAATTC** — é reconhecido pela enzima EcoRI, usada em engenharia genética;
- **ATG** — é o “códon de início” que indica onde começa uma proteína.

Buscar motifs é simplesmente procurar uma substring dentro de uma string maior. **Exemplo:**

```

1 DNA:    ATGCCGAATTCGAATTCATG
2 Motif:  GAATTC
3 Resultado: encontrado 2 vezes (posicoes 6 e 12)

```

Distância de Hamming

A distância de Hamming entre duas sequências de **mesmo tamanho** é o número de posições onde elas diferem. Quanto menor a distância, mais parecidas as sequências.

Exemplo:

```

1 Seq 1: ATGCCGT
2 Seq 2: ATCCCGT
3      || ||| (diferenca na posicao 3)
4 Distancia de Hamming = 1

```

Em biologia, isso é usado para detectar mutações entre indivíduos da mesma espécie.

2.2 Parte B — Conteúdos de Linguagem C

Variáveis e tipos de dados

Em C, toda variável deve ser declarada com seu **tipo**. Os principais tipos que vamos usar:

Tipo	Para que serve	Exemplo de uso no trabalho
int	Números inteiros	Tamanho da sequência, contagem de bases
float	Números com casas decimais	Conteúdo GC (percentual)
char	Um único caractere	Uma base de DNA ('A', 'T', etc.)
char[]	Vetor de caracteres (string)	A sequência de DNA inteira

Exemplo:

```

1 int    tamanho = 19;           // tamanho de uma sequencia
2 float  gc = 52.63;            // conteudo GC
3 char   primeiraBase = 'A';    // uma unica base
4 char   dna[200] = "ATGCCG..."; // sequencia completa

```

Operadores

Categoria	Operadores	No trabalho
Aritméticos	+ - * / %	Cálculo do GC, contagem
Relacionais	== != < > <= >=	Comparar bases, validar tamanho
Lógicos	&& !	Validar entradas múltiplas
Atribuição	= += -= ++ -	Incrementar contadores

Exemplo combinado:

```
1 if (base == 'G' || base == 'C') { // logico OR
2     contadorGC++;                // incremento
3 }
4 float percentual = (float)contadorGC / total * 100; // aritmetica
```

Estruturas condicionais: if e switch

O if/else permite executar trechos diferentes dependendo de uma condição:

```
1 if (gc > 50.0) {
2     printf("Sequencia rica em GC\n");
3 } else if (gc < 30.0) {
4     printf("Sequencia pobre em GC\n");
5 } else {
6     printf("Sequencia com GC normal\n");
7 }
```

O switch é uma alternativa mais limpa quando você precisa comparar uma variável com vários valores específicos. **Vamos usá-lo para tratar o menu principal e para identificar bases:**

```
1 switch (base) {
2     case 'A': contaA++; break;
3     case 'T': contaT++; break;
4     case 'C': contaC++; break;
5     case 'G': contaG++; break;
6     default: printf("Base invalida!\n");
7 }
```

Atenção: sem o break, o programa “cai” para o próximo case — um erro comum em iniciantes.

Estruturas de repetição: for, while, do-while

Os três laços fazem coisas parecidas, mas têm uso ideal em situações diferentes:

for — use quando souber **quantas vezes** vai repetir:

```
1 // Percorrer cada base da sequencia
2 for (int i = 0; i < tamanho; i++) {
3     printf("Base %d: %c\n", i, dna[i]);
4 }
```

while — use quando **não souber** de antemão quando vai parar:

```
1 // Ler caracteres do arquivo enquanto houver
2 while (fgets(linha, 200, arquivo) != NULL) {
3     processarLinha(linha);
4 }
```

do-while — garante que o bloco execute **pelo menos uma vez**. Ideal para menus:

```
1 int opcao;
2 do {
```

```

3     mostrarMenu();
4     scanf("%d", &opcao);
5     // ... processar opcao ...
6 } while (opcao != 5); // sai quando escolher "5 - Sair"

```

Funções e modularização

Uma função é um “bloco de código com nome” que pode ser chamado de várias partes do programa. Modularizar significa quebrar o programa em funções pequenas e bem definidas, em vez de ter tudo em uma main gigante.

Anatomia de uma função:

```

1 // |tipo de retorno| |nome| |parametros (entradas)|
2 float calcularGC(char *dna, int tamanho) {
3     int gc = 0;
4     for (int i = 0; i < tamanho; i++) {
5         if (dna[i] == 'G' || dna[i] == 'C') gc++;
6     }
7     return (float)gc / tamanho * 100;
8     // ^----- valor que sai da funcao
9 }
10
11 // Como chamar:
12 float resultado = calcularGC(minhaSequencia, 19);

```

Funções sem retorno usam void:

```

1 void exibirCabecalho() {
2     printf("== ANALISADOR DE DNA ==\n");
3 }

```

Por que modularizar? Código curto é mais fácil de testar, debugar e reaproveitar. Uma boa regra: **se uma função tem mais de 30 linhas, provavelmente deveria ser quebrada em funções menores.**

Vetores (arrays)

Vetor é uma coleção de variáveis do mesmo tipo armazenadas em sequência na memória. Em C, vetores têm **tamanho fixo** definido na declaração:

```

1 int   contagem[4];           // vetor de 4 inteiros (A, T, C, G)
2 char  dna[200];             // vetor de 200 caracteres (uma sequencia)
3 char  banco[10][200];       // 10 sequencias, cada uma com 200
                               // caracteres

```

Acesso por índice — importante: índices começam em 0!

```

1 char primeira = dna[0];     // primeira base
2 char ultima   = dna[18];    // 19a base (se tamanho = 19)
3 contagem[0] = 5;           // armazena 5 na primeira posicao

```

Strings

Em C, uma string é **simplesmente um vetor de char terminado com o caractere especial '\0'** (nulo). Isso é fundamental:

```
1 char dna[] = "ATGCC";
2
3 Memoria:  [ 'A' ][ 'T' ][ 'G' ][ 'C' ][ 'C' ][ '\0' ]
4           0      1      2      3      4      5
5
6 strlen(dna) retorna 5 (sem contar o '\0')
```

Funções úteis da <string.h>:

Função	O que faz
strlen(s)	Retorna o tamanho da string (sem o '\0')
strcpy(dest, origem)	Copia uma string para outra
strcmp(s1, s2)	Compara duas strings: retorna 0 se iguais
strcat(dest, origem)	Concatena origem no final de dest

Estruturas (struct)

struct permite agrupar dados de tipos diferentes em uma única “caixa” com nome. **Sem struct**, para armazenar 10 sequências precisamos de vários vetores paralelos:

```
1 char nomes[10][30];           // nome de cada sequencia
2 char dnas[10][200];           // DNA de cada sequencia
3 int tamanhos[10];             // tamanho de cada sequencia
4 // Problema: dados relacionados ficam espalhados!
```

Com struct, agrupamos tudo:

```
1 typedef struct {
2     char nome[30];
3     char dna[200];
4     int tamanho;
5     float gc;
6 } Sequencia;
7
8 Sequencia banco[10];           // vetor de 10 sequencias completas
9
10 // Acesso aos campos com o operador ponto (.):
11 banco[0].tamanho = 19;
12 strcpy(banco[0].nome, "Seq_Teste1");
13 printf("GC: %.2f\n", banco[0].gc);
```

Vantagem: todos os dados relacionados ficam juntos, fica mais fácil passar como parâmetro e organizar o código.

Ordenação de vetores

Ordenar significa reorganizar os elementos em alguma ordem (crescente ou decrescente). Existem vários algoritmos — vamos usar o **Bubble Sort** no trabalho.

Como funciona o Bubble Sort: percorre o vetor várias vezes, comparando elementos vizinhos e trocando-os de lugar quando estão fora de ordem. A cada “passagem”, o maior elemento “borbulha” até o final.

```

1 void ordenar(int v[], int n) {
2     int aux;
3     for (int i = 0; i < n - 1; i++) {
4         for (int j = 0; j < n - 1 - i; j++) {
5             if (v[j] > v[j + 1]) {           // fora de ordem?
6                 aux = v[j];                 // troca
7                 v[j] = v[j + 1];
8                 v[j + 1] = aux;
9             }
10        }
11    }
12 }

```

Passo a passo com vetor [4, 1, 3, 2]:

1. Compara 4 e 1 → troca → [1, 4, 3, 2]
2. Compara 4 e 3 → troca → [1, 3, 4, 2]
3. Compara 4 e 2 → troca → [1, 3, 2, 4] (4 chegou ao final)
4. Nova passagem: compara 1 e 3 → ok, 3 e 2 → troca → [1, 2, 3, 4]
5. Vetor ordenado!

Ponteiros

Um **ponteiro** é uma variável que guarda o **endereço de memória** de outra variável. Pense nele como “um indicador que aponta para onde a variável de verdade está armazenada”.

```

1 int x = 10;           // variavel comum
2 int *p = &x;         // ponteiro p recebe o ENDEREÇO de x
3                       // (o operador & significa "endereço de")
4
5 printf("%d", *p);     // imprime 10 - o asterisco acessa o VALOR
6                       // apontado (operador "conteudo de")
7
8 *p = 20;              // muda o valor de x via ponteiro
9 printf("%d", x);      // agora imprime 20!

```

Por que usar ponteiros? O uso mais importante é permitir que uma função **modifique** variáveis do código que a chamou. Compare:

```

1 // SEM ponteiro: a funcao recebe uma COPIA do valor
2 void dobrarErrado(int n) {
3     n = n * 2; // so muda a copia, nao o original
4 }
5
6 // COM ponteiro: a funcao recebe o ENDEREÇO do valor
7 void dobrarCerto(int *n) {
8     *n = *n * 2; // muda o valor original
9 }
10

```

```

11 int x = 5;
12 dobrarErrado(x);    printf("%d", x);    // imprime 5 (nao mudou!)
13 dobrarCerto(&x);    printf("%d", x);    // imprime 10 (mudou!)

```

Ponteiros e vetores: todo vetor em C é “visto” pelo C como um ponteiro para o seu primeiro elemento. Por isso, podemos passar um vetor para uma função e modificá-lo:

```

1 void contar(char *dna, int *resultado) {
2     *resultado = 0;
3     for (int i = 0; dna[i] != '\0'; i++) {
4         if (dna[i] == 'G') (*resultado)++;
5     }
6 }
7
8 int main() {
9     char meuDNA[] = "ATGCCG";
10    int qtd;
11    contar(meuDNA, &qtd);    // passa endereco de qtd
12    printf("G aparece %d vezes\n", qtd);
13    return 0;
14 }

```

Ponteiros para struct: usamos o operador -> em vez de .:

```

1 void mostrarGC(Sequencia *s) {
2     printf("Sequencia %s: GC = %.2f%%\n", s->nome, s->gc);
3     // Equivalente a (*s).nome e (*s).gc
4 }

```

Manipulação de arquivos

Arquivos permitem **persistir** dados — ou seja, salvá-los para usar depois que o programa fechar. Em C, usamos um **ponteiro para arquivo** (tipo FILE *).

Passos básicos:

1. Abrir o arquivo com `fopen`, indicando o **modo** (leitura, escrita, etc.).
2. Verificar se a abertura funcionou (`fopen` retorna NULL em caso de erro).
3. Ler/escrever com `fprintf`, `fscanf`, `fgets`, `fputs`.
4. **Fechar** o arquivo com `fclose` (importante para não perder dados).

Modos principais:

Modo	Significado
"r"	Leitura (read): falha se o arquivo não existir
"w"	Escrita (write): cria o arquivo ou apaga se já existir
"a"	Anexar (append): adiciona ao final, sem apagar

Exemplo completo — escrita:

```

1 FILE *arq = fopen("sequencias.txt", "w");
2 if (arq == NULL) {

```



```
3     printf("Erro: nao foi possivel abrir o arquivo!\n");
4     return;
5 }
6 fprintf(arq, "Seq_Testes1\n");
7 fprintf(arq, "ATGCCG\n");
8 fclose(arq);          // SEMPRE fechar
```

Exemplo completo — leitura:

```
1 FILE *arq = fopen("sequencias.txt", "r");
2 if (arq == NULL) {
3     printf("Arquivo nao encontrado.\n");
4     return;
5 }
6 char linha[200];
7 while (fgets(linha, 200, arq) != NULL) {
8     // remove o '\n' que fgets coloca no final
9     linha[strcspn(linha, "\n")] = '\0';
10    printf("Lido: %s\n", linha);
11 }
12 fclose(arq);
```

RESUMO RÁPIDO: O QUE PRECISO LEMBRAR

- Toda variável em C tem um **tipo** (int, float, char, ...).
- Strings em C são **vetores de char terminados com '\0'**.
- **for** para repetições com número conhecido, **while** para incerto, **do-while** para garantir execução pelo menos uma vez (ótimo para menus).
- **struct** agrupa dados relacionados em uma única “caixa”.
- Ponteiros permitem que funções **modifiquem** variáveis do código que as chamou (uso de * e &).
- Arquivos: **fopen** → verificar NULL → ler/escrever → **fclose**.

3 Divisão do Trabalho

Parte	Conteúdo	Peso
I	Básico: menu, cadastro, listagem, contagem de bases, conteúdo GC	6,0
II	Intermediário: struct, ordenação, busca de motif	2,0
III	Avançado: arquivos, ponteiros, comparação de sequências (Hamming)	2,0
TOTAL		10,0

PARTE I — BÁSICO (6,0 pontos)

Objetivo: construir as funcionalidades fundamentais com sequências de tamanho fixo (até 200 bases).

1.1. Menu principal (1,5 ponto)

Implementar menu que se repete até o usuário escolher “Sair”, usando `do-while` e `switch`.

Opções obrigatórias:

1. Cadastrar sequência
2. Listar sequências
3. Contar bases (A, T, C, G)
4. Calcular conteúdo GC
5. Sair

Esqueleto sugerido:

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define MAX_SEQ 10          // ate 10 sequencias
5  #define TAM_DNA 201        // ate 200 bases + '\0'
6  #define TAM_NOME 31        // nome ate 30 caracteres
7
8  int main() {
9      int opcao;
10     do {
11         printf("\n== ANALISADOR DE DNA ==\n");
12         printf("1 - Cadastrar sequencia\n");
13         printf("2 - Listar sequencias\n");
14         printf("3 - Contar bases\n");
15         printf("4 - Calcular conteudo GC\n");
16         printf("5 - Sair\n");
17         printf("Escolha: ");
18         scanf("%d", &opcao);
19         switch (opcao) {
20             case 1: cadastrar(); break;
21             case 2: listar(); break;
22             case 3: contarBases(); break;
23             case 4: calcularGC(); break;
24             case 5: printf("Encerrando...\n"); break;
25             default: printf("Opcao invalida!\n");
26         }
27     } while (opcao != 5);
28     return 0;
29 }
```

1.2. Cadastro de sequências (2,0 pontos)

Permitir cadastrar até 10 sequências usando vetores paralelos:

```

1 char nomes[MAX_SEQ][TAM_NOME]; // nome de cada sequencia
2 char dnas[MAX_SEQ][TAM_DNA]; // sequencia de DNA
3 int tamanhos[MAX_SEQ]; // tamanho de cada sequencia
4 int totalSeqs = 0; // quantas ja foram cadastradas

```

Validações:

- Só aceitar caracteres A, T, C, G (maiúsculas). Se houver caractere inválido, rejeitar a sequência e exibir mensagem.
- Limite de 10 sequências cadastradas.

Função de validação sugerida:

```

1 int sequenciaValida(char *dna) {
2     for (int i = 0; dna[i] != '\0'; i++) {
3         if (dna[i] != 'A' && dna[i] != 'T' &&
4             dna[i] != 'C' && dna[i] != 'G') {
5             return 0; // invalida
6         }
7     }
8     return 1; // valida
9 }

```

1.3. Listagem das sequências (0,8 ponto)

Listar todas as sequências cadastradas em formato simples:

ID	Nome	Tamanho	DNA
1	Seq_Testes1	19	ATGCCGTAACGGATTAGCC
2	Seq_Testes2	20	GCGCGCGCATATATCGCGCG

1.4. Contagem de bases (0,9 ponto)

Função que percorre uma sequência e conta quantas vezes cada base aparece:

```

1 void contarBases(char *dna, int *qtdA, int *qtdT,
2                 int *qtdC, int *qtdG) {
3     *qtdA = *qtdT = *qtdC = *qtdG = 0;
4     for (int i = 0; dna[i] != '\0'; i++) {
5         switch (dna[i]) {
6             case 'A': (*qtdA)++; break;
7             case 'T': (*qtdT)++; break;
8             case 'C': (*qtdC)++; break;
9             case 'G': (*qtdG)++; break;
10        }
11    }
12 }

```

Saída esperada:

```
1 Sequencia: Seq_Teste1 (19 bases)
2 A: 5      T: 4      C: 5      G: 5
```

1.5. Conteúdo GC (0,8 ponto)

CONTEXTO BIOLÓGICO

O **conteúdo GC** é o percentual de bases G e C em uma sequência. É uma medida usada em biologia para caracterizar diferentes organismos.

Fórmula: $GC\% = \frac{qtdG + qtdC}{tamanho} \times 100$

```
1 float calcularGC(char *dna, int tamanho) {
2     int gc = 0;
3     for (int i = 0; i < tamanho; i++) {
4         if (dna[i] == 'G' || dna[i] == 'C')
5             gc++;
6     }
7     return (float)gc / tamanho * 100;
8 }
```

Saída esperada:

```
1 Sequencia Seq_Teste1: GC = 52.63%
```

PARTE II — INTERMEDIÁRIO (2,0 pontos)

Objetivo: organizar o sistema com **struct**, ordenar sequências e buscar padrões.

2.1. Refatoração com struct (0,8 ponto)

Substituir os vetores paralelos por um vetor de **structs**:

```
1 typedef struct {
2     char   nome[31];
3     char   dna[201];
4     int     tamanho;
5     float  gc;           // conteudo GC calculado no cadastro
6 } Sequencia;
7
8 Sequencia banco[MAX_SEQ];
9 int totalSeqs = 0;
```

Adaptar **todas** as funções da Parte I para usar a **struct**.

2.2. Ordenação de sequências (0,7 ponto)

Adicionar a opção “**Ordenar sequências**” ao menu. Implementar **um** algoritmo de ordenação (o grupo escolhe entre Bolha, Seleção ou Inserção) que ordene as sequências por **tamanho crescente**.

Exemplo com Bubble Sort:

```
1 void ordenarPorTamanho(Sequencia *vet, int n) {
2     Sequencia aux;
3     for (int i = 0; i < n - 1; i++) {
4         for (int j = 0; j < n - 1 - i; j++) {
5             if (vet[j].tamanho > vet[j + 1].tamanho) {
6                 aux = vet[j];
7                 vet[j] = vet[j + 1];
8                 vet[j + 1] = aux;
9             }
10        }
11    }
12 }
```

2.3. Busca de motif (0,5 ponto)

CONTEXTO BIOLÓGICO

Um **motif** é um padrão curto de DNA com significado biológico. Localizar motifs é fundamental em biologia molecular.

Adicionar a opção “**Buscar motif**” ao menu. O usuário escolhe uma sequência e digita o padrão (ex.: GAATTC). O sistema deve dizer quantas vezes o padrão aparece e em que posições.

```
1 int buscarMotif(char *dna, char *padrao) {
```

```
2     int tamDna = strlen(dna);
3     int tamPad = strlen(padrao);
4     int qtd = 0;
5
6     for (int i = 0; i <= tamDna - tamPad; i++) {
7         int encontrou = 1;
8         for (int j = 0; j < tamPad; j++) {
9             if (dna[i + j] != padrao[j]) {
10                 encontrou = 0;
11                 break;
12             }
13         }
14         if (encontrou) {
15             printf("Encontrado na posicao %d\n", i + 1);
16             qtd++;
17         }
18     }
19     return qtd;
20 }
```

PARTE III — AVANÇADO (2,0 pontos)

Objetivo: adicionar persistência em arquivo e comparação entre sequências usando ponteiros.

3.1. Salvar e carregar arquivo (1,2 ponto)

Adicionar duas opções ao menu: “**Salvar em arquivo**” e “**Carregar do arquivo**”.

Formato do arquivo `sequencias.txt`: cada sequência ocupa duas linhas — primeira com o nome, segunda com o DNA:

```
1 Seq_Teste1
2 ATGCCGTAACGGATTAGCC
3 Seq_Teste2
4 GCGCGCGCATATATCGCGCG
5 Seq_Teste3
6 AAAATTTTAAAATTTTAAAAT
```

Função para salvar:

```
1 void salvarArquivo(Sequencia *banco, int total) {
2     FILE *arq = fopen("sequencias.txt", "w");
3     if (arq == NULL) {
4         printf("Erro ao abrir arquivo para escrita!\n");
5         return;
6     }
7     for (int i = 0; i < total; i++) {
8         fprintf(arq, "%s\n", banco[i].nome);
9         fprintf(arq, "%s\n", banco[i].dna);
10    }
11    fclose(arq);
}
```

```

12     printf("Salvas %d sequencias no arquivo.\n", total);
13 }

```

Função para carregar:

```

1  int carregarArquivo(Sequencia *banco) {
2      FILE *arq = fopen("sequencias.txt", "r");
3      if (arq == NULL) {
4          printf("Arquivo nao encontrado.\n");
5          return 0;
6      }
7      int total = 0;
8      while (total < MAX_SEQ &&
9              fgets(banco[total].nome, TAM_NOME, arq) != NULL &&
10             fgets(banco[total].dna, TAM_DNA, arq) != NULL) {
11          // Remove o '\n' do final
12          banco[total].nome[strcspn(banco[total].nome, "\n")] = '\0';
13          banco[total].dna[strcspn(banco[total].dna, "\n")] = '\0';
14          ;
15          banco[total].tamanho = strlen(banco[total].dna);
16          total++;
17      }
18      fclose(arq);
19      printf("Carregadas %d sequencias do arquivo.\n", total);
20      return total;
21 }

```

3.2. Comparação por distância de Hamming (0,8 ponto)

CONTEXTO BIOLÓGICO

A **distância de Hamming** entre duas sequências de **mesmo tamanho** é o número de posições onde as bases diferem. Quanto menor a distância, mais parecidas as sequências.

Exemplo: ATGCC vs ATCCC → distância = 1.

Adicionar a opção “**Comparar sequências**” ao menu. O usuário escolhe duas sequências e o sistema mostra a distância de Hamming entre elas. **Usar ponteiros** nos parâmetros da função:

```

1  // Recebe ponteiros para duas sequencias e retorna a distancia
2  // (ou -1 se os tamanhos forem diferentes).
3  int distanciaHamming(Sequencia *s1, Sequencia *s2) {
4      if (s1->tamanho != s2->tamanho) {
5          return -1;
6      }
7      int distancia = 0;
8      for (int i = 0; i < s1->tamanho; i++) {
9          if (s1->dna[i] != s2->dna[i]) {
10             distancia++;

```



```
11     }  
12 }  
13 return distancia;  
14 }
```

Saída esperada:

```
1 Comparando Seq_Testes1 vs Seq_Testes3...  
2 Tamanho: 19 bases  
3 Distancia de Hamming: 8  
4 Similaridade: 57.89%
```

4 Plano de Testes

Os grupos devem executar os testes abaixo e registrar os resultados no relatório final.

Dados de teste sugeridos

```

1 Seq_A: ATGCCGTAACGGATTAGCC (19 bases - mista)
2 Seq_B: GCGCGCGCATATATCGCGCG (20 bases - rica em GC)
3 Seq_C: AAAATTTTAAAATTTTAAAA (20 bases - rica em AT)
4 Seq_D: ATGCCGTAACGGATTAGCC (19 bases - idêntica a Seq_A)
5 Seq_E: ATGCCGTAACGGATTAGCT (19 bases - 1 diferença de Seq_A)

```

TESTES OBRIGATÓRIOS

#	Cenário	Entrada	Resultado Esperado
Parte I — Básico			
T01	Cadastrar sequência válida	Nome: Seq_A, DNA: ATGCCG...	"Cadastrada com sucesso"
T02	Cadastrar com base inválida	DNA: ATGXCG	"Erro: sequencia invalida"; não cadastra
T03	Atingir limite de cadastros	Cadastrar 11 sequências	10 cadastradas; 11 ^a recusada
T04	Listar sem cadastros	Antes de qualquer cadastro	"Nenhuma sequencia cadastrada"
T05	Listar com 3 sequências	Após 3 cadastros	Tabela com 3 linhas
T06	Contar bases de Seq_A	Seq_A (ATGCCGTAACGGATTAGCC)	A=5, T=4, C=5, G=5
T07	Contar bases de Seq_B	Seq_B (rica em GC)	A=2, T=2, C=8, G=8
T08	Contar bases de Seq_C	Seq_C (sem GC)	A=12, T=8, C=0, G=0
T09	GC de Seq_A (19 bases)	Seq_A	GC ≈ 52.63%
T10	GC de Seq_B (20 bases)	Seq_B	GC = 80.00%
T11	GC de Seq_C (sem GC)	Seq_C	GC = 0.00%
Parte II — Intermediário			
T12	struct mantém funcionalidades	Repetir T01–T11	Todos os testes anteriores passam

T13	Ordenar por tamanho crescente	Seq_A(19), Seq_B(20), Seq_C(20)	Seq_A primeiro, depois Seq_B/Seq_C
T14	Ordenar com 1 sequência	Apenas Seq_A	Não dá erro
T15	Buscar motif existente	Seq_A, motif GAT	1 ocorrência na posição 11
T16	Buscar motif inexistente	Seq_A, motif ZZZ	0 ocorrências
Parte III — Avançado			
T17	Salvar 3 sequências em arquivo	Após cadastrar 3	Arquivo <code>sequencias.txt</code> criado
T18	Carregar arquivo salvo	Abrir programa novamente e carregar	As 3 sequências reaparecem
T19	Carregar arquivo inexistente	Apagar arquivo e tentar carregar	“Arquivo nao encontrado”
T20	Hamming entre Seq_A e Seq_D (iguais)	Seq_A vs Seq_D	Distância = 0
T21	Hamming entre Seq_A e Seq_E (1 dif.)	Seq_A vs Seq_E	Distância = 1
T22	Hamming entre tamanhos diferentes	Seq_A (19) vs Seq_B (20)	Mensagem “tamanhos diferentes”

Como documentar os testes no relatório

Para cada teste executado, registrar em uma tabela simples:

Teste	Observações	Status
T01	_____	☐ OK ☐ Falhou
T02	_____	☐ OK ☐ Falhou
... repetir para T03 até T22 ...		

5 Entregáveis

1. **Código-fonte** (.c) em arquivo .zip nomeado TrabalhoDNA_GRUPO_X.zip.
2. **Relatório em PDF** (4 a 6 páginas) contendo:
 - Capa com identificação do grupo;
 - Breve descrição do sistema;
 - Explicação das três partes com pelo menos 1 trecho de código por parte;
 - Tabela de testes preenchida;

- Capturas de tela (mínimo 5): menu, cadastro, contagem, GC, motif;
- Divisão de tarefas entre os integrantes;
- Considerações finais.

3. **Vídeo de apresentação** (5 a 8 minutos), **todos os integrantes participando**, demonstrando o sistema funcionando e explicando o código.

6 Critérios de Avaliação

Critério	Descrição	Peso
Parte I (Básico)	Menu, cadastro com validação, listagem, contagem de bases, conteúdo GC.	6,0
Parte II (Intermediário)	struct, 1 algoritmo de ordenação, busca de motif.	2,0
Parte III (Avançado)	Salvar/carregar arquivo, comparação com Hamming usando ponteiros.	2,0
TOTAL		10,0

Penalidades

- Código não compila: **nota zero** na parte correspondente.
- Plágio entre grupos ou cópia da internet: **nota zero no trabalho inteiro**.
- Entrega atrasada: -1,0 por dia (limite de 3 dias).
- Vídeo sem participação de algum integrante: -1,0 para o integrante ausente.

7 Cronograma Sugerido (2 semanas)

Dia	Etapa	Atividades
1-2	Planejamento	Reunião do grupo, divisão de tarefas.
3-7	Parte I	Menu, cadastro, validação, listagem, contagem, GC. Executar T01-T11.
8-10	Parte II	Refatoração para struct, ordenação, motif. Executar T12-T16.
11-12	Parte III	Arquivo (salvar/carregar) e Hamming. Executar T17-T22.
13	Documentação	Relatório, capturas de tela, vídeo.
14	Entrega	Testes finais e upload.

8 Orientações Técnicas

Compilação

```
1 gcc -Wall -o analisador_dna analisador_dna.c
2 ./analisador_dna
```

Boas práticas

- Indentação consistente (4 espaços ou Tab);
- Cada função em uma parte separada do código, com nome descritivo;
- Comente o que cada função faz (entrada, processamento, saída);
- Teste o código **frequentemente**, não deixe tudo para o final.
- código fonte em <https://github.com/alex-marino/dna>

Bom trabalho! Em caso de dúvidas, procurem o professor.
